Benchmarking de Inferência do YOLO Quantizado em MCU e SBC

Desempenho e eficiência energética do YOLOX-Nano para detecção de EPIs em *Edge AI*

Projeto de Pesquisa — Trilha PTT6

Filipe Matias
Guilherme Lucas

Orientador: Prof. Herbert Oliveira Rocha

Mestrado Profissional em Computação Aplicada (PPCA) — Universidade Federal de Roraima

Disciplina: Arquitetura e Organização de Computadores

Outubro de 2026

Resumo

Este projeto compara, de forma sistemática e reprodutível, o desempenho e a eficiência energética da inferência de um mesmo detector de objetos quantizado — o YOLOX-Nano em int8 — em dois extremos do espectro *Edge AI*: o microcontrolador ESP32 (Xtensa LX6) e o computador de placa única Raspberry Pi Zero 2 W. A aplicação é a detecção de Equipamentos de Proteção Individual (EPIs) em canteiros de obra, com o conjunto Ultralytics Construction-PPE. Seguindo a prática dos benchmarks de referência da área, a acurácia do modelo quantizado (mAP) é condição de validade e é reportada lado a lado com a latência por fase, o throughput e a energia por inferência, estimada por um modelo de potência por modo de operação aplicado aos tempos medidos. A hipótese principal afirma que o SBC tem maior throughput, mas energia por inferência igual ou superior à da MCU. O desenho prevê piloto para definir repetições e aquecimento, coleta em blocos independentes, estatística robusta (Mann-Whitney U, δ de Cliff e intervalos por *bootstrap*) e análise de sensibilidade do modelo de potência.

Palavras-chave: Edge AI; TinyML; quantização int8; YOLOX; benchmarking; eficiência energética.

1 Introdução

A inferência de redes neurais diretamente em dispositivos de borda reduz latência, tráfego de rede e exposição de dados, mas impõe restrições severas de memória, computação e energia. Dois extremos se destacam: os **microcontroladores (MCU)**, de consumo muito baixo e memória da ordem de centenas de kilobytes, e os **computadores de placa única (SBC)**, que executam um sistema operacional completo e oferecem mais desempenho a um custo energético maior.

O monitoramento do uso de EPIs em canteiros de obra torna essa escolha concreta: a inspeção precisa operar continuamente, muitas vezes com alimentação restrita, e o detector precisa ser confiável. A literatura de TinyML concentra-se em tarefas de classificação; para detectores,

faltam comparações diretas e reprodutíveis do *mesmo* modelo quantizado executado nas duas classes de plataforma, com acurácia, latência e energia avaliadas em conjunto.

1.1 Pergunta de pesquisa e hipóteses

Pergunta: como se comparam latência, throughput e energia por inferência entre uma MCU (ESP32) e um SBC (Raspberry Pi Zero 2 W) ao executar o mesmo YOLOX-Nano quantizado em int8 na detecção de EPIs, com a acurácia do modelo como condição de validade?

H1: o SBC apresenta throughput maior, mas energia por inferência igual ou superior, dado o seu consumo estático mais alto.

H0: não há diferença de throughput nem de energia por inferência entre as plataformas.

H2: no SBC, a quantização int8 reduz latência e energia por inferência em relação ao modelo em ponto flutuante (fp32), ao custo de uma perda de acurácia que é quantificada.

1.2 Objetivos

Geral: comparar sistematicamente o desempenho e a eficiência energética da inferência de um modelo YOLOX-Nano quantizado em uma MCU e em um SBC representativos do espectro de *Edge AI*.

Específicos:

1. obter um artefato YOLOX-Nano int8 para detecção de EPIs e quantificar sua acurácia em relação ao modelo fp32 no conjunto de teste;
2. implementar um pipeline de inferência funcionalmente idêntico nas duas plataformas e verificar a equivalência das saídas;
3. medir latência por fase, throughput e estimar a energia por inferência em cada plataforma, sob protocolo declarado;
4. testar H1 e H2 com estatística robusta e análise de sensibilidade do modelo de energia;
5. publicar código, artefatos e dados brutos para reprodução.

1.3 Contribuição

Uma caracterização reprodutível, acurácia × latência × energia, de um mesmo detector quantizado executado nos dois extremos da borda, com dados e código abertos e com a análise de *break-even* energético que indica em que condições a MCU é mais eficiente que o SBC.

1.4 Vínculo com a disciplina

O estudo mobiliza desempenho e a Lei de Amdahl (tempos entre arquiteturas e ganho de paralelismo nos quatro núcleos do SBC), a hierarquia de memória (SRAM e PSRAM do ESP32 versus DRAM do SBC) e E/S.

2 Fundamentação e trabalhos relacionados

Métricas de avaliação. Na literatura de benchmarking de inferência, energia e latência não são resultados válidos isoladamente. O MLPerf Inference associa a cada tarefa uma acurácia-alvo

(tipicamente 99% da acurácia de referência em ponto flutuante), que o sistema precisa atingir para que o desempenho conte [1]; o MLPerf Tiny mede acurácia, latência e energia e exige uma acurácia mínima por modelo [2]. Trabalhos de detecção voltados a energia reportam acurácia e energia como eixos de um trade-off, e não como um número composto [3, 4]; o detector do tipo YOLO quantizado para MCU mais próximo deste estudo reporta mAP ao lado da energia por inferência [5]. Surveys de TinyML e de quantização tratam a avaliação como multiobjetivo [6, 7]. Para a janela de medição, seguimos a separação em pré-inferência, inferência e pós-inferência e o uso do EDP [8].

Quantização. A quantização int8 com aritmética inteira [9] é o padrão para inferência embarcada, e o TensorFlow Lite Micro é o runtime de referência em MCUs [10]. Quando bem executada, a quantização pós-treino mantém a queda de mAP de detectores abaixo de cerca de 1 ponto percentual [11]; a avaliação compara o mAP int8 e o fp32 no mesmo conjunto de teste [12]. Conjuntos de calibração da ordem de uma centena de amostras costumam bastar para estimar as faixas de ativação [13], mas ativações não-ReLU (sigmoide/Swish) podem degradar a acurácia quantizada [12]. Em MCUs, a implantação de redes quantizadas e o custo do ponto flutuante foram caracterizados por Novac et al. [14].

Detector e treinamento. O YOLOX é um detector *anchor-free* com cabeça desacoplada; sua variante Nano tem cerca de 0,91 M parâmetros e 1,08 GFLOPs a 416×416 [15]. As 300 épocas do YOLOX referem-se a treino do zero no COCO [15]; a partir de pesos pré-treinados, o ajuste fino requer bem menos iterações [16], e estudos de detecção de EPI com YOLO escolhem o número de épocas empiricamente, em geral com parada antecipada [17, 18]. Adotamos a parada pela validação [19] e as recomendações de relato e reprodutibilidade [20–22]. O número de repetições é definido pela variabilidade observada [23].

3 Materiais

3.1 Plataformas

Tabela 1: Plataformas comparadas (fator independente).

	MCU	**SBC**
Dispositivo	ESP32 (Xtensa LX6, dois núcleos, FPU escalar)	Raspberry Pi Zero 2 W (Cortex-A53, quatro núcleos)
Clock	240 MHz, sem escalonamento dinâmico nem *light-sleep*	1 GHz, governor `performance`
Memória	520 KB de SRAM + PSRAM externa	512 MB de LPDDR2
Runtime	TensorFlow Lite Micro (int8), um núcleo	LiteRT/TFLite (int8 e fp32), 4 threads
Alimentação	USB 5 V; rádio desligado	fonte 5 V/2,5 A

3.2 Modelo

YOLOX-Nano inicializado com os pesos COCO e ajustado uma única vez ao conjunto de EPIs (Seção 4.4), exportado para ONNX e convertido para TFLite em fp32 e em int8 *full-integer*

(Seção 4.5). O mesmo arquivo int8, com hash SHA-256 registrado, é executado nas duas plataformas; o fp32 serve de referência de acurácia e é executado no SBC.

3.3 Conjunto de dados

Ultralytics Construction-PPE [24] (AGPL-3.0), com 11 classes: *helmet*, *gloves*, *vest*, *boots*, *goggles*, *none*, *Person*, *no_helmet*, *no_goggle*, *no_gloves* e *no_boots*, nas partições oficiais. A classe de interesse principal é o colete (*vest*). O arquivo utilizado é identificado por seu hash SHA-256.

Tabela 2: Construction-PPE: imagens por partição e presença da classe *vest*.

Partição	Imagens	Instâncias *vest*	Imagens com *vest*
Treino	1132	1283	837
Validação	143	171	109
Teste	141	178	111

4 Metodologia experimental

4.1 Etapas

O trabalho segue três etapas com critério de saída: (1) **preparação e validação do modelo** — treino, quantização, avaliação de acurácia e verificação de equivalência nas duas plataformas; (2) **medição** — coleta de latência por fase e derivação de energia; (3) **análise** — estatística, sensibilidade e entrega. Nenhuma medição final ocorre antes de a etapa anterior estar concluída.

4.2 Variáveis

Tabela 3: Variáveis do experimento.

Tipo	**Variáveis**
Independentes	Plataforma ∈ {MCU, SBC} (H1); precisão ∈ {int8, fp32} no SBC (H2)
Dependentes	mAP@50 e mAP@50:95 (teste); AP@50 do colete; t_{pre}, t_{inf}, t_{post} e latência t_{total} (ms); throughput (inf/s); energia por inferência E (mJ); inferências/J; EDP (mJ·ms)
Controladas	artefato (hash), resolução comum r, ordem de canais BGR, limiares de pós-processamento, as mesmas 50 imagens de entrada, clock e governor, threads, rádio, alimentação, ambiente
Covariáveis	temperatura da CPU, frequência e *throttling* (SBC), temperatura ambiente, número de detecções, imagem, bloco e ordem de execução

4.3 Métricas e critério de qualidade

Para cada combinação {plataforma × precisão} reportam-se, lado a lado: mAP@50 e mAP@50:95 no conjunto de teste, com intervalo de confiança de 95% por *bootstrap* de imagens [25]; AP@50 do colete; latência por fase; throughput; energia estimada; inferências/J e EDP. Não se usa métrica composta escalar. A acurácia é condição de validade: define-se a retenção $\rho = \mathrm{mAP}_{\mathrm{int8}}/\mathrm{mAP}_{\mathrm{fp32}}$, reportada como variável contínua, com alvo de referência $\rho \geq 0{,}99$ [1]. Como o mesmo artefato

int8 é executado nas duas plataformas, a acurácia é idêntica entre elas (verificada pela equivalência das saídas), e H1 compara as plataformas sob acurácia controlada; H2 compara as precisões no SBC como trade-off entre acurácia e energia.

4.4 Protocolo de treinamento

Ajuste fino a partir dos pesos COCO do YOLOX-Nano, a 416×416, com o número de épocas tratado como hiperparâmetro: teto declarado, cronograma do YOLOX (aquecimento inicial; Mosaic e MixUp desligados nas últimas épocas) [15], mAP de validação calculado a cada época, parada antecipada por paciência [19] e checkpoint final escolhido pelo melhor mAP de validação. Ao menos uma execução com semente fixa e curva de validação completa; havendo tempo, uma segunda semente, reportando a faixa [22]. Reportam-se taxa de aprendizado e cronograma, tamanho de lote, aumentações, épocas efetivas, semente e hardware [20]. A ordem de canais é BGR, a do treino original do YOLOX, em toda a cadeia.

4.5 Quantização

Quantização pós-treino *full-integer* em int8 (pesos por canal, ativações por tensor), calibrada com 750 imagens do conjunto de treino distribuídas pelas classes, com o mesmo pré-processamento da inferência (*letterbox* 416×416, BGR, sem normalização). O artefato int8 é aceito para a etapa de medição após a avaliação de acurácia; caso não atinja acurácia aceitável para a aplicação, substitui-se o modelo por outra versão quantizada da família YOLO, mantendo todo o protocolo.

4.6 Avaliação de acurácia

A acurácia é medida com o protocolo COCO sobre o conjunto de teste (141 imagens, 1.251 anotações), para os modelos fp32 e int8, com o mesmo pré e pós-processamento dos dispositivos: *letterbox* com preenchimento 114, entrada BGR, decodificação da cabeça do YOLOX (passos 8, 16 e 32), pontuação = objetividade $\times$ probabilidade de classe, NMS por classe. Para o mAP usam-se confiança 0,001 e NMS 0,65; os limiares de operação (0,25 e 0,45) servem apenas à demonstração. Reportam-se mAP@50, mAP@50:95, AP por classe e o intervalo por *bootstrap* (1.000 reamostras de imagens).

4.7 Pipeline e janela de medição

A latência cobre a janela completa pré-processamento + inferência + pós-processamento, e as três fases são registradas separadamente. As entradas são as mesmas 50 imagens do conjunto de teste, convertidas uma vez em tensores brutos $r \times r \times 3$ uint8 em BGR, idênticos byte a byte nas duas plataformas; o pré-processamento medido é a cópia e a quantização para o tensor de entrada, e o pós-processamento é a decodificação e o NMS, com a mesma lógica nas duas plataformas. A resolução r é comum às duas plataformas: a maior de $\{416, 320, 256, 224, 160\}$ que caiba na memória do ESP32 e permita ao menos 30 inferências por bloco em até 60 min.

4.8 Ingestão das imagens

No ESP32, o computador envia cada imagem pela serial USB-UART em um quadro com cabeçalho e CRC32; o firmware a armazena em RAM, confirma o recebimento e só então abre a janela cronometrada; tempos e detecções retornam pela serial depois da janela. No Raspberry Pi, as imagens são lidas do disco para a memória antes do laço. Nas duas plataformas a janela começa com a imagem já em memória, e o custo de transferência fica fora da medição.

4.9 Instrumentação

Os tempos vêm de relógios monotônicos de alta resolução (`time.perf_counter_ns()` no SBC; `esp_timer_get_time()` no MCU). Não há E/S dentro da janela: as medidas ficam em memória e são gravadas entre blocos. Temperatura, frequência e *throttling* são lidos fora da janela. Cada execução gera um CSV com uma linha por inferência e um arquivo de metadados (versões, configurações, hashes e ambiente).

4.10 Experimentos

Tabela 4: Experimentos.

Id	Descrição e saída
E0	Acurácia fp32 × int8 no conjunto de teste: mAP@50, mAP@50:95, AP do colete, retenção ρ (IC)
E1	Equivalência funcional: mesmas 50 entradas no PC, SBC e MCU; mesmas classes e número de detecções, IoU $\geq 0{,}95$ e $\lvert\Delta$ pontuação$\rvert \leq 1$ passo de quantização
E2	Piloto: viabilidade de r no MCU; 100 inferências por plataforma; aquecimento W e número N
E3	Benchmark principal: 3 blocos por plataforma, $N_{final} = \max(30, N)$; int8 nas duas plataformas e fp32 no SBC
E3b	SBC com 1, 2 e 4 threads: fração paralelizável pela Lei de Amdahl
E4	Throughput sustentado em 60 s de laço fechado e deriva térmica
E5	Energia derivada, sensibilidade, *break-even* e energia por ciclo

Aquecimento e número de repetições. Em janelas de 10 inferências, descartam-se as iniciais até a variação relativa da mediana entre janelas consecutivas ficar abaixo de 2% em duas janelas seguidas (piso de 5). Com o coeficiente de variação (CV) de t_{total} medido no piloto,

$$N = \left\lceil \left(\frac{1{,}96\,\mathrm{CV}}{\varepsilon} \right)^2 \right\rceil, \qquad \varepsilon = 1\%, \qquad N_{final} = \max(30, N), \tag{1}$$

podendo N diferir entre plataformas.

Estrutura de um bloco (E3). Religar o dispositivo e aguardar 2 min ocioso (registrar temperatura e *throttling*); executar W inferências de aquecimento (marcadas e excluídas); executar $\lceil N_{final}/3 \rceil$ inferências cronometradas, percorrendo as 50 entradas em ordem fixa; gravar CSV e metadados; aguardar o resfriamento (temperatura de volta a $\pm 2\,^{\circ}$C da inicial no SBC, ou 5 min no MCU). Os blocos são intercalados entre plataformas.

4.11 Modelo de energia

A energia é estimada a partir dos tempos medidos por fase e da potência total do dispositivo em cada modo:

$$E_i = P_{pre}\, t_{pre,i} + P_{inf}\, t_{inf,i} + P_{post}\, t_{post,i}, \tag{2}$$

com os parâmetros da Tabela 5. No SBC, pré e pós-processamento usam o modo de um núcleo, e a inferência, o modo correspondente ao número de threads.

Tabela 5: Parâmetros do modelo de potência (nominal e faixa de sensibilidade).

Plataforma	**Modo**	**Nominal**	**Faixa**	**Fonte**
SBC (placa, 5 V)	ocioso	0,60 W	0,50–1,00 W	[26, 27]
SBC	ativo, 1 núcleo	1,00 W	0,75–1,30 W	[27]
SBC	ativo, 4 núcleos	2,30 W	1,85–2,90 W	[27, 28]
MCU (chip, 3,3 V)	ativo, 240 MHz	0,224 W	0,099–0,224 W	[29]
MCU	acréscimo da placa	0 W	0–0,25 W	parâmetro declarado

Com potência constante por modo, E é uma transformação linear dos tempos, e a incerteza do modelo é tratada por sensibilidade. A condição de *break-even* é

$$E_{SBC} \geq E_{MCU} \iff \frac{P_{SBC}}{P_{MCU}} \geq \frac{t_{MCU}}{t_{SBC}}, \tag{3}$$

e reporta-se se a razão de potências necessária cai dentro da faixa plausível. Como os valores do SBC são de placa e os do MCU de chip, H1 é avaliada também no pior caso para o MCU. Para uma frequência de inspeção f, reporta-se ainda a energia por ciclo $E_{ciclo}(f) = P_{ativo}\, t + P_{ocioso}(1/f - t)$, válida para $t < 1/f$, com $f \in \{0{,}2;\ 0{,}5;\ 1\}$ Hz.

4.12 Análise estatística

Por plataforma e precisão, reportam-se mediana e IQR, média ± desvio-padrão e CV das inferências válidas de E3. A comparação usa o teste de Mann-Whitney U (bilateral para latência e unilateral nas direções de H1), o δ de Cliff e intervalos de confiança de 95% por *bootstrap* BCa (10.000 reamostras) para a diferença e a razão das medianas. H1 é avaliada como teste de interseção–união, com $\text{throughput}_{SBC} > \text{throughput}_{MCU}$ e $E_{SBC} \geq E_{MCU}$, cada parte a $\alpha = 0{,}05$; a parte energética é avaliada no valor nominal e nos extremos da Tabela 5. H2 é avaliada pela retenção ρ e pela redução de latência e energia do int8 em relação ao fp32 no SBC. Outliers não são removidos; a remoção por Tukey (1,5×IQR) entra apenas como sensibilidade. Verificam-se o efeito de bloco (Kruskal-Wallis) e a autocorrelação (ACF de *lag* 1). Uma análise secundária usa apenas t_{inf}, isolando o núcleo de inferência das diferenças de implementação do pré e do pós-processamento.

5 Ameaças à validade

Tabela 6: Principais ameaças e mitigação.

Ameaça	**Mitigação**
Energia estimada, não medida fisicamente	mesmo método nas duas plataformas; premissas registradas; sensibilidade e *break-even*
Parâmetros de potência do SBC oriundos de medições técnicas publicadas fora de periódicos	faixa ampla de sensibilidade; conclusão testada nos extremos
Escopos distintos de potência (placa × chip)	acréscimo de placa do MCU como parâmetro; H1 no pior caso para o MCU
Perda de acurácia na quantização	avaliação obrigatória no teste; retenção ρ reportada; substituição do modelo se inaceitável
Ordem de canais inconsistente entre treino e inferência	BGR fixo em treino, calibração, avaliação e dispositivos
Throttling térmico no SBC	governor fixo, dissipador, pausas; bloco com *throttling* é repetido
Pré e pós-processamento em linguagens diferentes	mesma lógica e limiares; análise secundária só com t_{inf}
Resolução limitada pela memória do MCU	mesma r nas duas plataformas
Ingestão por serial (MCU) × disco (SBC)	transferência fora da janela; janela inicia com a imagem em memória
Conjunto de teste pequeno (141 imagens)	intervalos por *bootstrap*; AP por classe
Autocorrelação e deriva	blocos independentes, ordem registrada, ACF e efeito de bloco

6 Cronograma

Tabela 7: Cronograma (15 semanas).

Semanas	**Atividades**
1–2	Revisão de literatura e protocolo experimental
3–5	Treino, quantização, avaliação de acurácia (E0), pipelines nas duas plataformas e equivalência (E1)
6–9	Piloto (E2), benchmark principal (E3, E3b) e throughput sustentado (E4)
10–11	Energia derivada e sensibilidade (E5); análise estatística
12–13	Redação do relatório
14–15	Vídeo de demonstração e entrega final

7 Entregáveis

Repositório versionado com licença MIT; README de reprodutibilidade; artefatos do modelo com hash; dados brutos em CSV e metadados; vídeo de demonstração (3–5 min) nas duas plataformas, mostrando a ingestão de imagens e as detecções; relatório técnico conclusivo. Após a disciplina: dataset com DOI (Zenodo) e submissão a evento regional.

Referências

[1] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William

Chou, Ramesh Chukka, Cody Coleman, Sam Davis, Pan Deng, Greg Diamos, Jared Duke, Dave Fick, J. Scott Gardner, Itay Hubara, Sachin Idgunji, Thomas B. Jablin, Jeff Jiao, Tom St. John, Pankaj Kanwar, David Lee, Jeffery Liao, Anton Lokhmotov, Francisco Massa, Peng Meng, Paulius Micikevicius, Colin Osborne, Gennady Pekhimenko, Arun Tejusve Raghunath Rajan, Dilip Sequeira, Ashish Sirasao, Fei Sun, Hanlin Tang, Michael Thomson, Frank Wei, Ephrem Wu, Lingjie Xu, Koichi Yamada, Bing Yu, George Yuan, Aaron Zhong, Peizhao Zhang, and Yuchen Zhou. MLPerf inference benchmark. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020. doi: 10.1109/isca45697.2020.00045. Verificado: texto completo (arXiv:1911.02549) confirma Tabela I ("99

[2] Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kiraly, Pietro Montino, David Kanter, Sebastian Ahmed, Danilo Pau, Urmish Thakker, Antonio Torrini, Pete Warden, Jay Cordaro, Giuseppe Di Guglielmo, Javier Duarte, Stephen Gibellini, Videet Parekh, Honson Tran, Nhan Tran, Niu Wenxu, and Xu Xuesong. MLPerf tiny benchmark. *arXiv preprint arXiv:2106.07597*, 2021. doi: 10.48550/arXiv.2106.07597. Verificado: texto completo confirma literalmente "Each model has a minimum accuracy that must be met for the score to be valid.".

[3] Peng Tu, Xu Xie, Guo Ai, Yuexiang Li, Yawen Huang, and Yefeng Zheng. FemtoDet: An object detection baseline for energy versus performance tradeoffs. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. doi: 10.1109/ICCV51070.2023.01225.

[4] Floran De Putter, Sherif Eissa, and Henk Corporaal. POQ: Is there a Pareto-optimal quantization strategy for deep neural networks? *IEEE Access*, 13:81434–81449, 2025. doi: 10.1109/ACCESS.2025.3567046.

[5] Julian Moosmann, Marco Giordano, Christian Vogt, and Michele Magno. TinyissimoYOLO: A quantized, low-memory footprint, TinyML object detection network for low power microcontrollers. In *2023 IEEE 5th International Conference on Artificial Intelligence Circuits and Systems (AICAS)*, 2023. doi: 10.1109/aicas57966.2023.10168657. Verificado: texto completo confirma reporte conjunto de mAP (ex.: 45

[6] Partha Pratim Ray. A review on TinyML: State-of-the-art and prospects. *Journal of King Saud University - Computer and Information Sciences*, 2022. doi: 10.1016/j.jksuci.2021.11.019. Verificado: metadados Crossref confirmados; resumo (OpenAlex) confirma enquadramento multiobjetivo (acuracia, processamento, confiabilidade).

[7] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. A survey of quantization methods for efficient neural network inference, 2021. CONSOLIDADO: o mesmo trabalho foi citado no .bib da metrica principal como capitulo de livro (10.1201/9781003162810-13, "Low-Power Computer Vision", 2022) e no .bib de PTQ como preprint arXiv (2103.13630, 2021). Ambos os DOIs resolvem corretamente para a mesma autoria/titulo; mantida aqui uma unica entrada (versao arXiv, aberta) para evitar duplicidade no .bib final do artigo.

[8] Noronha et al. Benchmarking energy and latency in TinyML. *arXiv preprint arXiv:2505.15622*, 2025. Completar a lista de autores antes da versão final.

[9] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. doi: 10.1109/cvpr.2018.00286. Verificado: texto completo confirma Tabela 4.4 (SSD-MobileNet/COCO, 100

[10] Robert David, Jared Duke, Advait Jain, Vijay Janapa Reddi, Nat Jeffries, Jian Li, Nick Kreeger, Ian Nappier, Meghna Natraj, Shlomi Regev, Rocky Rhodes, Tiezhen Wang, and Pete Warden. TensorFlow lite micro: Embedded machine learning on TinyML systems. *arXiv preprint arXiv:2010.08678*, 2020. doi: 10.48550/arXiv.2010.08678. Verificado por metadados (OpenAlex/Crossref); texto completo nao lido nesta auditoria.

[11] Hao Wu, Patrick Judd, Xiaojie Zhang, Mikhail Isaev, and Paulius Micikevicius. Integer quantization for deep learning inference: Principles and empirical evaluation, 2020. Verificado: texto completo confirma Tabela 7 – para os tres detectores avaliados (Faster R-CNN, Mask R-CNN, RetinaNet), deltas de PTQ de -0.35

[12] Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart van Baalen, and Tijmen Blankevoort. A white paper on neural network quantization, 2021. Verificado: texto completo confirma o mecanismo per-tensor/per-channel de magnitudes heterogeneas, E uma passagem adicional (nao usada pelo relatorio de PTQ) diretamente relevante ao achado E0: "Other more complex activation functions, such as sigmoid or Swish... require more dedicated support... This can have a big impact on the accuracy of quantized model- - suporta a hipotese de que ativacoes SiLU/Swish (nao-ReLU) no YOLOX-Nano padrao contribuem para o colapso observado, independente da concatenacao da saida (refutada pela evidencia E0 do projeto).

[13] Raghuraman Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper, 2018. Verificado: texto completo confirma literalmente "about 100 mini-batches are sufficient for the estimates of the ranges of the activation to converge"e a Tabela 2 (quantizacao so de pesos) mostrando MobileNetV1 com quantizacao assimetrica per-layer caindo a 0.001 de acuracia top-1 (nivel de chance) vs 0.709 em ponto flutuante.

[14] Pierre-Emmanuel Novac, Ghouthi Boukli Hacene, Alain Pegatoquet, Benoît Miramond, and Vincent Gripon. Quantization and deployment of deep neural networks on microcontrollers. *Sensors*, 21(9):2984, 2021. doi: 10.3390/s21092984.

[15] Zheng Ge, Songtao Liu, Feng Wang, Zeming Li, and Jian Sun. YOLOX: Exceeding YOLO series in 2021. *arXiv preprint arXiv:2107.08430*, 2021. doi: 10.48550/arXiv.2107.08430. Verificado: texto completo confirma literalmente "300 epochs with 5 epochs warm-up", cosine LR, SGD, e "close it [Mosaic/MixUp] for the last 15 epochs"; busca textual confirma ausencia de "fine-tun", "custom dataset", "small dataset"no PDF.

[16] Kaiming He, Ross Girshick, and Piotr Dollár. Rethinking ImageNet pre-training. In *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. doi: 10.1109/iccv.2019.00502. Verificado: texto completo confirma literalmente as tres citacoes (schedule 2x/ 24 epocas vs 6x/ 72 epocas; "90k iterations (1x)"/"180k iterations (2x)"do Detectron/Mask R-CNN; robustez com "10

[17] Mungunkhuyag Otgonbold, Munkhjargal Gochoo, Fady Alnajjar, Luqman Ali, Tan-Hsu Tan, Jun-Wei Hsieh, and Ping-Yang Chen. SHEL5K: An extended dataset and benchmarking for safety helmet detection. *Sensors*, 2022. doi: 10.3390/s22062315. Verificado por metadados (Crossref); afirmacao sobre "1000 epocas (YOLOv3) / 500 (YOLOv4/v5/YOLOR), determinado empiricamente"consistente com o resumo (OpenAlex), texto completo nao relido nesta auditoria.

[18] Luqman Ali, Fady Alnajjar, Medha Mohan Ambali Parambil, Mohamad I. Younes, Zahid I. Abdelhalim, and Hamad Aljassmi. Development of YOLOv5-based real-time smart monitoring system for increasing lab safety awareness in educational institutions. *Sensors*, 2022. doi: 10.3390/s22228820. Verificado por metadados (Crossref); citacao sobre "300 epochs... early stopping to prevent model overfitting"consistente com o conteudo do artigo.

[19] Lutz Prechelt. Early stopping — but when? In *Neural Networks: Tricks of the Trade*, Lecture Notes in Computer Science. 1998. doi: 10.1007/3-540-49430-8_3. Verificado por metadados (Crossref); texto completo nao lido nesta auditoria.

[20] Jesse Dodge, Suchin Gururangan, Dallas Card, Roy Schwartz, and Noah A. Smith. Show your work: Improved reporting of experimental results. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019. doi: 10.18653/v1/d19-1224. Verificado por metadados (Crossref); texto completo nao lido nesta auditoria.

[21] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Lariviere, Alina Beygelzimer, Florence d'Alche Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 22(164):1–20, 2021. doi: 10.48550/arXiv.2003.12206. Verificado: texto completo confirma a descricao do checklist de reprodutibilidade e do programa do NeurIPS 2019.

[22] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018. doi: 10.1609/aaai.v32i1.11694. Verificado por metadados (Crossref); texto completo nao lido nesta auditoria.

[23] Aleksander Maricq, Dmitry Duplyakin, Ivo Jimenez, Carlos Maltzahn, Ryan Stutsman, and Robert Ricci. Taming performance variability. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 409–425, 2018.

[24] Ultralytics. Construction-PPE detection dataset. https://docs.ultralytics.com/datasets/detect/construction-ppe/, 2025. Licença AGPL-3.0. Acesso em: out. 2026.

[25] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Springer US, 1993. doi: 10.1007/978-1-4899-4541-9.

[26] CNX Software. A deep dive into Raspberry Pi Zero 2 W's power consumption. https://www.cnx-software.com/2021/12/09/raspberry-pi-zero-2-w-power-consumption/, 2021.

[27] Jeff Geerling. Disabling cores to reduce the Pi Zero 2 W's power consumption by half. https://www.jeffgeerling.com/blog/2021/disabling-cores-reduce-pi-zero-2-ws-power-consumption-half/, 2021.

[28] Tom's Hardware. Raspberry Pi Zero 2 W review: The long awaited sequel. https://www.tomshardware.com/reviews/raspberry-pi-zero-2-w-review, 2021.

[29] Espressif Systems. *ESP32 Series Datasheet, versão 5.3*. Tabela 4-2, Power Consumption by Power Modes.